Assignment 3

Week 3

6733172621 Patthadon Phengpinij

Collaborators. Claude (for LaTeX styling and grammar checking)

Problem 1. Finding Eigenvectors by Hand

This is a tedious exercise. But I want you to go through to practice the skill of finding eigenvectors. We can also measure the electron spin in $\hat{y}$ direction by setting up an external magnetic field in the $\hat{y}$ direction. It is given that the corresponding operator is

$$\sigma_y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}.$$

Again, we are using the $|0\rangle\,/\,|1\rangle$ basis. Find the eigenvalues and eigenvectors in the $|0\rangle\,/\,|1\rangle$ basis.

Solution. To find the eigenvalues, we solve the characteristic equation $\det(\sigma_y - \lambda I) = 0$:

$$\det \begin{pmatrix} -\lambda & -i \\ i & -\lambda \end{pmatrix} = \lambda^2 - (-i)(i) = \lambda^2 - 1 = 0$$

which gives $\lambda_1 = 1$ and $\lambda_2 = -1$.
Eigenvector for $\lambda_1 = 1$. Solve $(\sigma_y - I)\,\vec{v} = \vec{0}$:

$$\begin{pmatrix} -1 & -i \\ i & -1 \end{pmatrix} \begin{pmatrix} v_1 \\ v_2 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$$

The first row gives $-v_1 - iv_2 = 0$, i.e. $v_1 = -iv_2$. Setting $v_1 = 1$ gives $v_2 = i$, so the unnormalized eigenvector is $\begin{pmatrix} 1 \\ i \end{pmatrix}$. Normalizing (norm $= \sqrt{1^2 + |i|^2} = \sqrt{2}$):

$$|\lambda_1 = 1\rangle = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ i \end{pmatrix} = \frac{1}{\sqrt{2}} \left(|0\rangle + i\,|1\rangle \right)$$

Eigenvector for $\lambda_2 = -1$. Solve $(\sigma_y + I)\,\vec{v} = \vec{0}$:

$$\begin{pmatrix} 1 & -i \\ i & 1 \end{pmatrix} \begin{pmatrix} v_1 \\ v_2 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$$

The first row gives $v_1 - iv_2 = 0$, i.e. $v_1 = iv_2$. Setting $v_1 = 1$ gives $v_2 = -i$, so the unnormalized eigenvector is $\begin{pmatrix} 1 \\ -i \end{pmatrix}$. Normalizing:

$$|\lambda_2 = -1\rangle = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ -i \end{pmatrix} = \frac{1}{\sqrt{2}} \left(|0\rangle - i\,|1\rangle \right)$$

Check.

$$\sigma_y\,|\lambda_1 = 1\rangle = \frac{1}{\sqrt{2}} \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix} \begin{pmatrix} 1 \\ i \end{pmatrix} = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ i \end{pmatrix} = |\lambda_1 = 1\rangle$$

$$\sigma_y\,|\lambda_2 = -1\rangle = \frac{1}{\sqrt{2}} \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix} \begin{pmatrix} 1 \\ -i \end{pmatrix} = -\frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ -i \end{pmatrix} = -\,|\lambda_2 = -1\rangle$$

both of which confirm $\sigma_y\,|\lambda_k\rangle = \lambda_k\,|\lambda_k\rangle$, as expected.

Problem 2. Finding Eigenvectors Using Python

Use Google Colab to check your answer. What you need is the function to find the eigenvectors and eigenvalues. The following code first builds the matrix and then find the eigenvectors and eigenvalues:

```
sigmay = np.array([[0, -1j], [1j, 0]])
print(sigmay)

eigenvalues, eigenvectors = np.linalg.eig(sigmay)
print(eigenvalues)
print(eigenvectors)
```

Solution. Running the provided code in Google Colab prints

$$\text{eigenvalues} \approx \begin{pmatrix} 1 \\ -1 \end{pmatrix}, \qquad \text{eigenvectors} \approx \begin{pmatrix} -0.7071i & 0.7071 \\ 0.7071 & -0.7071i \end{pmatrix}$$

where each column of the eigenvector matrix is the eigenvector for the eigenvalue in the corresponding position.

1. The first column, $\begin{pmatrix} -0.7071i \\ 0.7071 \end{pmatrix} = -i \cdot \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ i \end{pmatrix}$, is the hand-calculated eigenvector for $\lambda_1 = 1$ up to the global phase factor $-i$, which is physically irrelevant since an eigenvector is only defined up to a nonzero scalar multiple.
2. The second column, $\begin{pmatrix} 0.7071 \\ -0.7071i \end{pmatrix} \approx \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ -i \end{pmatrix}$, matches the hand-calculated eigenvector for $\lambda_2 = -1$ exactly.

Problem 3. Pauli Matrix Multiplication by Hand and Using Python

Use hand calculation to show that $\sigma_y^2 = I$. Then verify with Colab using the following codes. Note that `np.matmul` is for matrix multiplication.

```
sigmay = np.array([[0, -1j], [1j, 0]])

print(sigmay)
print(np.matmul(sigmay, sigmay))
```

Solution. By hand:

$$\sigma_y^2 = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix} \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix} = \begin{pmatrix} (-i)(i) & 0 \\ 0 & (i)(-i) \end{pmatrix} = \begin{pmatrix} -i^2 & 0 \\ 0 & -i^2 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} = I$$

Verifying with the provided Python code, the output is

$$\sigma_y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}$$

$$\sigma_y^2 = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$$

which matches the hand-calculated result $\sigma_y^2 = I$.

Problem 4. Pauli Matrix Multiplication by Hand and Using Python

Repeat the previous question for $\sigma_z^2 = I$ using both hand calculation and Colab.

Solution. By hand, with $\sigma_z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$:

$$\sigma_z^2 = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}\begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} = \begin{pmatrix} 1^2 & 0 \\ 0 & (-1)^2 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} = I$$

Verifying with Colab:

```
sigmaz = np.array([[1, 0], [0, -1]])

print(sigmaz)
print(np.matmul(sigmaz, sigmaz))
```

which prints $\sigma_z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$ and $\sigma_z^2 = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$, matching the hand-calculated result.

Problem 5. Proof of Hermitianity

Show that σ_y is Hermitian. Verify using Colab.

$\Big($*Hints:* find σ_y^{*T} and show that it is the same as $\sigma_y\Big)$

Solution. By hand, the conjugate transpose of σ_y is obtained by transposing and then conjugating every entry:

$$\sigma_y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix} \implies \sigma_y^T = \begin{pmatrix} 0 & i \\ -i & 0 \end{pmatrix} \implies \sigma_y^{*T} = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix} = \sigma_y$$

Since $\sigma_y^{*T} = \sigma_y$, σ_y is Hermitian.
Verifying with Colab:

```
sigmay_dagger = sigmay.conj().T

print(sigmay_dagger)
print(np.allclose(sigmay_dagger, sigmay))
```

which prints $\begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}$, identical to σ_y, and `True` for the equality check, confirming σ_y is Hermitian.

Problem 6. Operation of Paul Matrix on Bra and Ket

Find $\sigma_y |0\rangle$, $\langle 0| \sigma_y$. Are they the same? Why? Verify using Colab. You can use `np.dot` for matrix-vector multiplication. The following shows how to do $\sigma_y |0\rangle$.

```
zerospin = np.array([[1], [0]])

print(zerospin.shape)
print(np.dot(sigmay, zerospin))
```

Solution. First, $\sigma_y |0\rangle$:

$$\sigma_y |0\rangle = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 0 \\ i \end{pmatrix} = i |1\rangle$$

Next, $\langle 0| \sigma_y$, where $\langle 0| = \begin{pmatrix} 1 & 0 \end{pmatrix}$:

$$\langle 0| \sigma_y = \begin{pmatrix} 1 & 0 \end{pmatrix} \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix} = \begin{pmatrix} 0 & -i \end{pmatrix} = -i \langle 1|$$

They are **not** the same: $\sigma_y |0\rangle = i |1\rangle$ while $\langle 0| \sigma_y = -i \langle 1|$, which differ by a sign on the imaginary coefficient. This is expected because, since σ_y is Hermitian (Problem 5), $\langle 0| \sigma_y = (\sigma_y |0\rangle)^{*^T}$ is the *conjugate* transpose, not just the transpose, of $\sigma_y |0\rangle$; as $\sigma_y |0\rangle = i |1\rangle$ has a nonzero imaginary coefficient, conjugation flips its sign.
Verifying with Colab:

```
zerospin = np.array([[1], [0]])
print(zerospin.shape)

sigma_y_zero = np.dot(sigmay, zerospin)
print(sigma_y_zero)

bra_zero = zerospin.conj().T
bra_zero_sigma_y = np.dot(bra_zero, sigmay)
print(bra_zero_sigma_y)
```

which prints $\begin{pmatrix} 0 \\ i \end{pmatrix}$ for $\sigma_y |0\rangle$ and $\begin{pmatrix} 0 & -i \end{pmatrix}$ for $\langle 0| \sigma_y$, matching the hand-calculated result.

APPENDIX 1

Assignment 03 | Code Solution

assignment-03-code

August 28, 2026

1 Assignment 03

1.1 Import External Libraries

[1]:
```
import numpy as np
import sympy as sp

from IPython.display import display, Math
```

1.2 2. Finding Eigenvectors Using Python

[2]:
```
sigmay = np.array([[0, -1j],[1j, 0]])

display(Math(f"\\sigma_y = {sp.latex(sp.Matrix(sigmay))}"))

print("---------------")

eigenvalues, eigenvectors = np.linalg.eig(sigmay)
eigenvalues = np.round(eigenvalues, 4)
eigenvectors = np.round(eigenvectors, 4)

display(Math(f"\\text{{Eigenvalues: }} {sp.latex(sp.Matrix(eigenvalues))}"))
display(Math(f"\\text{{Eigenvectors: }} {sp.latex(sp.Matrix(eigenvectors))}"))
```

$$\sigma_y = \begin{bmatrix} 0 & -1.0i \\ 1.0i & 0 \end{bmatrix}$$

$$\text{Eigenvalues: } \begin{bmatrix} 1.0 \\ -1.0 \end{bmatrix}$$

$$\text{Eigenvectors: } \begin{bmatrix} -0.7071i & 0.7071 \\ 0.7071 & -0.7071i \end{bmatrix}$$

1

1.3 3. Pauli Matrix Multiplication by Hand and Using Python

[3]:
```
sigmay = np.array([[0, -1j], [1j, 0]])

display(Math(f"\\sigma_y = {sp.latex(sp.Matrix(sigmay))}"))
display(Math(f"\\sigma_y^2 = {sp.latex(sp.Matrix(np.matmul(sigmay, sigmay)))}"))
```

$$\sigma_y = \begin{bmatrix} 0 & -1.0i \\ 1.0i & 0 \end{bmatrix}$$

$$\sigma_y^2 = \begin{bmatrix} 1.0 & 0 \\ 0 & 1.0 \end{bmatrix}$$

1.4 4. Pauli Matrix Multiplication by Hand and Using Python

[4]:
```
sigmaz = np.array([[1, 0], [0, -1]])

display(Math(f"\\sigma_z = {sp.latex(sp.Matrix(sigmaz))}"))
display(Math(f"\\sigma_z^2 = {sp.latex(sp.Matrix(np.matmul(sigmaz, sigmaz)))}"))
```

$$\sigma_z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$

$$\sigma_z^2 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

1.5 5. Proof of Hermitianity

[5]:
```
sigmay_dagger = sigmay.conj().T
display(Math(f"\\sigma_y^{{*^T}} = {sp.latex(sp.Matrix(sigmay_dagger))}"))

print("---------------")

print(f"Check if Sigma Y is Hermitian: {np.allclose(sigmay_dagger, sigmay)}")
```

$$\sigma_y^{*^T} = \begin{bmatrix} 0 & -1.0i \\ 1.0i & 0 \end{bmatrix}$$

```
---------------
Check if Sigma Y is Hermitian: True
```

1.6 6. Operation of Paul Matrix on Bra and Ket

[6]:
```
zerospin = np.array([[1], [0]])
print("Dimension of zerospin:\n", zerospin.shape)

print("---------------")

sigma_y_zero = np.dot(sigmay, zerospin)
display(Math(f"\\sigma_y \\vert 0 \\rangle = {sp.latex(sp.
  ↪Matrix(sigma_y_zero))}"))

bra_zero = zerospin.conj().T
bra_zero_sigma_y = np.dot(bra_zero, sigmay)
display(Math(f"\\langle 0 \\vert \\sigma_y = {sp.latex(sp.
  ↪Matrix(bra_zero_sigma_y))}"))
```

```
Dimension of zerospin:
 (2, 1)
---------------
```

$$\sigma_y \vert 0 \rangle = \begin{bmatrix} 0 \\ 1.0i \end{bmatrix}$$

$$\langle 0 \vert \sigma_y = \begin{bmatrix} 0 & -1.0i \end{bmatrix}$$
